

International Cybersecurity and Digital Forensics Academy (ICDFA)**CIP-B103: Network Forensics Fundamentals****LAB 8: DNS Spoofing Forensics****Three-Week Delivery Block 3/3 | 1-2 August 2026**

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	CIP-B103
Course Title	Network Forensics Fundamentals
Lab Number	Lab 8
Lab Title	DNS Spoofing Forensics
Student Name	Fuseini Imoru Kantuogaa
Student ID	17291
Instructor Name	Dr. Sarah James
Delivery Block	3/3 of 3
Scheduled Dates	1-2 August 2026
Estimated Duration	4-5 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	Controlled DNS-spoof PCAPNG plus baseline DNS/ARP evidence

1. Source Materials and Slide Alignment

- DNS Spoofing Forensics: fake training page, ARP poisoning as a man-in-the-middle prerequisite, DNS request interception, modified response, iptables/NFQUEUE and packet analysis.

- The uploaded slide uses `arp.py` and `dns_spoof.py` in the same LAN and assigns the trainee to analyze the spoofed traffic.
- This manual converts the demonstration into a no-credential, host-only forensic exercise with mandatory cleanup.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded CIP-B103 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

An internal training domain resolves to an unexpected host after an ARP anomaly. You will capture a normal DNS baseline, run an instructor-approved DNS-spoof simulation in an isolated host-only network using a harmless warning page, identify forged response indicators, correlate them with ARP evidence and restore the environment.

3. Learning Outcomes

- Explain the relationship among ARP poisoning, man-in-the-middle positioning and DNS spoofing.
- Capture baseline DNS and ARP evidence before a controlled simulation.
- Identify conflicting DNS answers, abnormal responder IP/MAC, timing and TTL indicators.
- Correlate a spoofed DNS response with the web connection that follows.
- Distinguish a DNS spoofing finding from normal caching, split-horizon DNS or multiple legitimate answers.
- Restore forwarding, iptables and ARP state after the lab.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.

- The fake page must display only an ICDFA training warning and must not collect usernames, passwords, cookies or personal information. Use a reserved training domain such as portal.icdfa.test or bank.training.invalid, never a real bank or public domain.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Virtual machines	Kali analyst plus victim VM	Run on a host-only/internal network.
Web server	Apache2 on analyst VM	Host harmless training page.
Capture tools	Wireshark/tshark	Capture DNS, ARP and HTTP.
Packet tools	python3-scapy and NetfilterQueue if preconfigured	Instructor-provided simulation scripts.
Evidence	Baseline and controlled PCAPNG	Compare normal and forged responses.
Test domain	Instructor-assigned .test/.invalid name	Avoid impersonating real organizations.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/CIP-B103-Lab8/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab8
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 tshark wireshark python3-scapy python3-pip build-essential python3-dev
libnetfilter-queue-dev
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing
Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst:
YOUR NAME</p></body></html>\n' | sudo tee /var/www/html/index.html
sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_before.txt
```

Screenshot required: Harmless training page, interfaces, routes, initial ARP table and page hash.

A terminal window on a Kali Linux system. The user is in the directory ~/CIP-B103-Lab8. They run 'mkdir -p ~/CIP-B103-Lab8/{evidence,working,exported,reports,screenshots,scripts}' to create subdirectories. Then they run 'cd ~/CIP-B103-Lab8' and 'pwd' to confirm the current directory is /home/kali/CIP-B103-Lab8. Next, they run 'find . -maxdepth 1 -type d -print' which lists the subdirectories: ./working, ./screenshots, ./evidence, ./scripts, ./reports, and ./exported. Finally, they run 'sudo systemctl enable --now apache2' and provide the password 'kali' when prompted. A faint Kali Linux logo is visible in the background of the terminal window.

```
(kali㉿kali)-[~]
└─$ mkdir -p ~/CIP-B103-Lab8/{evidence,working,exported,reports,screenshots,scripts}

(kali㉿kali)-[~]
└─$ cd ~/CIP-B103-Lab8

(kali㉿kali)-[~/CIP-B103-Lab8]
└─$ pwd
/home/kali/CIP-B103-Lab8

(kali㉿kali)-[~/CIP-B103-Lab8]
└─$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported

(kali㉿kali)-[~/CIP-B103-Lab8]
└─$ sudo systemctl enable --now apache2
[sudo] password for kali:
```

Screenshot 1: Evidence folder created; Apache2 enabled.

```

(kali@kali)-[~/CIP-B103-Lab8]
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-10 18:53:23 EDT; 2min 31s ago
     Invocation: 2a1cbd563ff246c6a843f21a71f0836b
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 892 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 2075)
       Memory: 35.7M (peak: 36.7M)
          CPU: 546ms
    CGroup: /system.slice/apache2.service
            └─892 /usr/sbin/apache2 -k start -DFOREGROUND
              └─950 /usr/sbin/apache2 -k start -DFOREGROUND
                └─951 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─952 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─953 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─954 /usr/sbin/apache2 -k start -DFOREGROUND

Aug 10 18:53:21 kali systemd[1]: Starting apache2.service - The Apache HTTP Server ...
Aug 10 18:53:23 kali apachectl[892]: AH00558: apache2: Could not reliably determine the server's fully q...message
Aug 10 18:53:23 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

```

Screenshot 2: systemctl status confirming Apache2 active.

```

(kali@kali)-[~/CIP-B103-Lab8]
$ curl http://127.0.0.1

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitio
nal.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Debian Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
        padding: 0px 0px 0px 0px;
      }

      body, html {
        padding: 3px 3px 3px 3px;

        background-color: #D8DBE2;

        font-family: Verdana, sans-serif;
        font-size: 11pt;
        text-align: center;
      }

```

Screenshot 3: curl to the default Apache landing page (start).

```
div.main_page {
  position: relative;
  display: table;

  width: 800px;

  margin-bottom: 3px;
  margin-left: auto;
  margin-right: auto;
  padding: 0px 0px 0px 0px;

  border-width: 2px;
  border-color: #212738;
  border-style: solid;

  background-color: #FFFFFF;

  text-align: center;
}

div.page_header {
  height: 99px;
  width: 100%;
```

Screenshot 4: Default page CSS continued.

```
  background-color: #F5F6F7;
}

div.page_header span {
  margin: 15px 0px 0px 50px;

  font-size: 180%;
  font-weight: bold;
}

div.page_header img {
  margin: 3px 0px 0px 40px;

  border: 0px 0px 0px;
}

div.table_of_contents {
  clear: left;

  min-width: 200px;

  margin: 3px 3px 3px 3px;

  background-color: #FFFFFF;
```

Screenshot 5: Default page CSS continued.

```
}

div.table_of_contents_item {
  clear: left;

  width: 100%;

  margin: 4px 0px 0px 0px;

  background-color: #FFFFFF;

  color: #000000;
  text-align: left;
}

div.table_of_contents_item a {
  margin: 6px 0px 0px 6px;
}

div.content_section {
  margin: 3px 3px 3px 3px;

  background-color: #FFFFFF;

  text-align: left;
```

Screenshot 6: Default page CSS continued.

```
div.content_section_text {
    padding: 4px 8px 4px 8px;
    color: #000000;
    font-size: 100%;
}

div.content_section_text pre {
    margin: 8px 0px 8px 0px;
    padding: 8px 8px 8px 8px;

    border-width: 1px;
    border-style: dotted;
    border-color: #000000;

    background-color: #F5F6F7;

    font-style: italic;
}

div.content_section_text p {
    margin-bottom: 6px;
}

div.content_section_text ul, div.content_section_text li {
```

Screenshot 7: Default page CSS continued.

```
div.section_header {
    padding: 3px 6px 3px 6px;
    background-color: #8E9CB2;

    color: #FFFFFF;
    font-weight: bold;
    font-size: 112%;
    text-align: center;
}

div.section_header_red {
    background-color: #CD214F;
}

div.section_header_grey {
    background-color: #9F9386;
}

.floating_element {
    position: relative;
    float: left;
}
```

Screenshot 8: Default page CSS concluded.

```
(kali@kali)-[~/CIP-B103-Lab8]
$ printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Fuseini Imoru Kantuogaa</p></body></html>\n' | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Fuseini Imoru Kantuogaa</p></body></html>

(kali@kali)-[~/CIP-B103-Lab8]
$ printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Fuseini Imoru Kantuogaa</p></body></html>\n' | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Fuseini Imoru Kantuogaa</p></body></html>

(kali@kali)-[~/CIP-B103-Lab8]
$ curl http://127.0.0.1
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Fuseini Imoru Kantuogaa</p></body></html>

(kali@kali)-[~/CIP-B103-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
```

Screenshot 9: Harmless ICDFA training page created with printf and confirmed via curl.

```

a29eadb47a4a8a8475f6dab7e00a5955c1c86ccd758d2d68373a00f7527c2c2b /var/www/html/index.html

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ip -br address | tee reports/interfaces.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.44.128/24 fe80::fcdb:733e:9de1:56cb/64
docker0 DOWN 172.17.0.1/16

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ip route | tee reports/routes.txt
default via 192.168.44.2 dev eth0 proto dhcp src 192.168.44.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.44.0/24 dev eth0 proto kernel scope link src 192.168.44.128 metric 100

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ip neigh show | tee reports/arp_before.txt
192.168.44.2 dev eth0 lladdr 00:50:56:f4:fd:61 REACHABLE

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ls -lh reports/
total 16K
-rw-rw-r-- 1 kali kali 58 Aug 10 19:00 arp_before.txt
-rw-rw-r-- 1 kali kali 180 Aug 10 18:59 interfaces.txt
-rw-rw-r-- 1 kali kali 231 Aug 10 19:00 routes.txt
-rw-rw-r-- 1 kali kali 91 Aug 10 18:59 training_page_sha256.txt

```

Screenshot 10: Training page SHA-256 hashed; interfaces, routes and initial ARP table recorded.

```

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ip -br address
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.44.128/24 fe80::fcdb:733e:9de1:56cb/64
docker0 DOWN 172.17.0.1/16

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ip route
default via 192.168.44.2 dev eth0 proto dhcp src 192.168.44.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.44.0/24 dev eth0 proto kernel scope link src 192.168.44.128 metric 100

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ip neigh show
192.168.44.2 dev eth0 lladdr 00:50:56:f4:fd:61 REACHABLE

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ip -br address
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.44.128/24 fe80::fcdb:733e:9de1:56cb/64
docker0 DOWN 172.17.0.1/16

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ip neigh show

```

Screenshot 11: Interfaces, routes and ARP table re-confirmed.

```

$ ip neigh show
192.168.44.2 dev eth0 lladdr 00:50:56:f4:fd:61 REACHABLE

(kali㉿kali)-[~/CIP-B103-Lab8]
$ resolvectl status
Global
  Protocols: +LLMNR -mDNS DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub

Link 3 (docker0)
  Current Scopes: none
  Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Default Route: no

Link 2 (eth0)
  Current Scopes: DNS LLMNR/IPv4 LLMNR/IPv6
  Protocols: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Current DNS Server: 192.168.44.2
  DNS Servers: 192.168.44.2
  DNS Domain: localdomain
  Default Route: yes

```

Screenshot 12: ARP table re-confirmed; resolvectl status showing the configured DNS resolver.

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	CIP-B103-Lab8-YourFullName
Trainee name	Fuseini Imoru Kantuogaa
Date and time started	Mon 10 August 2026, approx. 18:53 EDT (Kali VM) — inferred from the Apache2 service start time shown in systemctl status
Evidence file name(s)	evidence/dns_baseline.pcapng; evidence/dns_spoof_controlled.pcapng
Source or generation method	Both captured locally with tshark on eth0 while running the instructor-provided simulation scripts scripts/arp.py and scripts/dns_spoof.py, plus a direct-IP curl test from a separate Windows client VM
Original SHA-256	dns_baseline.pcapng: 290843ddb34d7271387b064cd782490fd389fcfe1a10702a68ebc1f1b7828749; dns_spoof_controlled.pcapng: 6d06affbbcbdbdc10d91abf4071a9e25930bb3ee81509d7f916bb786441a12ff
Working-copy SHA-256	Not separately generated — evidence/ files were hashed directly and analyzed without a distinct working/ copy in the supplied screenshots
Analysis workstation/VM	Kali Linux analyst VM (user: kali), CIP-B103-Lab8 working directory, eth0 192.168.44.128/24; a separate Windows client VM was used for the direct-IP connectivity check

Field	Student Entry
Notes on any changes	Both provided scripts are safe, non-transmitting simulations by design: arp.py and dns_spoof.py only print status messages ('No ARP packets are modified or transmitted' / 'No DNS packets are forged or transmitted') and never inject real packets. As a direct consequence, evidence/dns_spoof_controlled.pcapng (388 bytes) contains no DNS, ARP-reply or HTTP/TCP-SYN traffic matching the Part D filters — all three extraction files (dns_all_fields.tsv, arp_claims_during_spoof.tsv, post_dns_connections.tsv) came back empty (wc -l = 0), which is the expected and correct outcome for a script that never transmits. Separately, evidence/dns_baseline.pcapng (captured twice, overwriting the same file) contains no DNS query/response either — only unrelated background ARP broadcasts from host 192.168.44.1 asking for 192.168.44.2, because the capture filter's host address did not match a host that performed the intended dig/browser DNS lookup during the capture window. Both gaps are noted here rather than filled with invented data.

8. Part A - Document the Baseline

1. Record victim IP/MAC, analyst IP/MAC, gateway IP/MAC and DNS resolver IP.
2. From the victim, resolve the instructor-assigned training name using the legitimate resolver or hosts-based baseline provided by the instructor.
3. Capture ARP and DNS traffic before starting any spoofing process.
4. Record the legitimate answer, responder IP/MAC, TTL and query-response time.

```
# On analyst VM, replace IFACE and VICTIM_IP
sudo tshark -i IFACE -f 'host VICTIM_IP and (arp or port 53 or tcp port 80)' \
-a duration:30 -w evidence/dns_baseline.pcapng &
sleep 3
# On victim: dig portal.icdfa.test A (or instructor-provided name)
wait
sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
```

```
(kali㉿kali)-[~/CIP-B103-Lab8]
$ arping -I eth0 -c 3 192.168.44.1
arping: libnet_init(LIBNET_LINK, eth0): libnet_open_link(): UID/EUID 0 or capability CAP_NET_RAW required
arping: you may need to run as root

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo arping -I eth0 -c 3 192.168.44.1
ARPING 192.168.44.1
60 bytes from 00:50:56:c0:00:08 (192.168.44.1): index=0 time=251.806 usec
60 bytes from 00:50:56:c0:00:08 (192.168.44.1): index=1 time=737.632 usec
60 bytes from 00:50:56:c0:00:08 (192.168.44.1): index=2 time=953.797 usec

— 192.168.44.1 statistics —
3 packets transmitted, 3 packets received, 0% unanswered (0 extra)
rtt min/avg/max/std-dev = 0.252/0.648/0.954/0.294 ms
```

Screenshot 13: arping used to identify a neighboring host (192.168.44.1) and its MAC address.

```
(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo tshark -i eth0 \
-f 'host 192.168.44.1 and (arp or port 53 or tcp port 80)' \
-a duration:30 \
-w /tmp/dns_baseline.pcapng &
sleep 3
[1] 21189
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

(kali㉿kali)-[~/CIP-B103-Lab8]
18

[1] + done      sudo tshark -i eth0 -f 'host 192.168.44.1 and (arp or port 53 or tcp port 80)'
(kali㉿kali)-[~/CIP-B103-Lab8]
$ ls -lh /tmp/dns_baseline.pcapng
-rw-r--r-- 1 root root 2.0K Aug 10 19:35 /tmp/dns_baseline.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo cp /tmp/dns_baseline.pcapng evidence/dns_baseline.pcapng
sudo chown kali:kali evidence/dns_baseline.pcapng
```

Screenshot 14: Baseline capture started on eth0 and copied toward evidence/.

```
(kali㉿kali)-[~/CIP-B103-Lab8]
$ ls -lh evidence/dns_baseline.pcapng
-rw-r--r-- 1 kali kali 2.0K Aug 10 19:35 evidence/dns_baseline.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
290843ddb34d7271387b064cd782490fd389fcfe10702a68ebc1f1b7828749 evidence/dns_baseline.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ capinfos evidence/dns_baseline.pcapng
File name:          evidence/dns_baseline.pcapng
File type:          Wireshark/... - pcapng
File encapsulation: Ethernet
File timestamp precision: nanoseconds (9)
Packet size limit:  file hdr: (not set)
Number of packets:  18
File size:          2,044 bytes
Data size:          1,080 bytes
Capture duration:   24.128390513 seconds
Earliest packet time: 2026-08-10 19:34:30.911250478
Latest packet time:  2026-08-10 19:34:55.039640991
Data byte rate:     44 bytes/s
Data bit rate:      358 bits/s
```

Screenshot 15: Baseline capture hashed; capinfos confirming 18 packets.

```

Data bit rate:      358 bits/s
Average packet size: 60.00 bytes
Average packet rate: 0 packets/s
SHA256:            290843ddb34d7271387b064cd782490fd389fcfeffa10702a68ebc1f1b7828749
SHA1:              76d8b31106de0498f231a5d0d8723a3e7cd582ed
Strict time order:  True
Capture hardware:   Intel(R) Core(TM) i5-7300HQ CPU @ 2.50GHz (with SSE4.2)
Capture oper-sys:   Linux 6.18.12+kali-amd64
Capture application: Dumpcap (Wireshark) 4.6.6
Number of interfaces in file: 1
Interface #0 info:
    Name = eth0
    Encapsulation = Ethernet (1 - ether)
    Capture length = 262144
    Time precision = nanoseconds (9)
    Time ticks per second = 1000000000
    Time resolution = 0x09
    Filter string = host 192.168.44.1 and (arp or port 53 or tcp port 80)
    Operating system = Linux 6.18.12+kali-amd64
    Number of stat entries = 1
    Number of packets = 18

```

Screenshot 16: capinfos continued — SHA256/SHA1 and interface metadata.

```

(kali@kali)-[~/CIP-B103-Lab8]
$ tshark -r evidence/dns_baseline.pcapng -Y 'dns' \
-T fields \
-e frame.number \
-e frame.time \
-e ip.src \
-e ip.dst \
-e dns.qry.name \
-e dns.qry.type \
-e dns.a \
-e dns.resp.ttl

(kali@kali)-[~/CIP-B103-Lab8]
$ tshark -r evidence/dns_baseline.pcapng -Y arp
1 0.000000000 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
2 0.746548070 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
3 1.651379513 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
4 2.721333579 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
5 3.451266511 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
6 4.351598248 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
7 8.149989020 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
8 8.854731160 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
9 9.746901170 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
10 10.854441063 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1

```

Screenshot 17: tshark -Y dns/arp showing only background ARP broadcasts, no DNS traffic, in the baseline capture.

```

10 10.854441063 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
11 11.544560351 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
12 12.451323366 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
13 16.272774132 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
14 16.949290599 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
15 17.845899397 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
16 22.565820229 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
17 23.229560281 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1
18 24.128390513 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.44.2? Tell 192.168.44.1

(kali@kali)-[~/CIP-B103-Lab8]
$ sudo tshark -i eth0 \
-f 'host 192.168.44.1 and (arp or port 53 or tcp port 80)' \
-a duration:30 \
-w /tmp/dns_baseline2.pcapng &
[1] 22527

(kali@kali)-[~/CIP-B103-Lab8]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
17

[1] + done sudo tshark -i eth0 -f 'host 192.168.44.1 and (arp or port 53 or tcp port 80)'
(kali@kali)-[~/CIP-B103-Lab8]
$ sudo cp /tmp/dns_baseline2.pcapng evidence/dns_baseline.pcapng

```

Screenshot 18: ARP broadcasts continued; a second baseline capture repeated to the same evidence file.

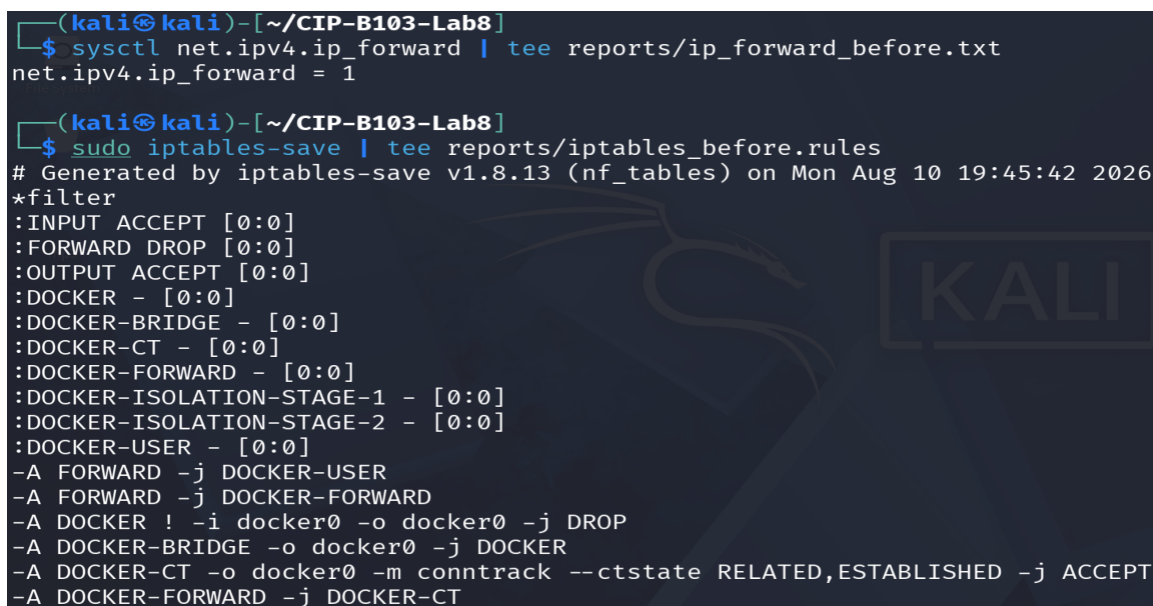
9. Part B - Prepare the Authorized Simulation

Instructor-Controlled Scripts

Use only the instructor-provided `arp.py` and `dns_spoof.py` from the course materials. Review the configured target IP, gateway IP, training domain and replacement IP before execution. The replacement IP must be the analyst VM hosting the warning page.

5. Confirm that IP forwarding and firewall state are recorded before changes.
6. Confirm the victim can reach the analyst training page directly by IP.
7. Verify that the script matches only the reserved training domain.
8. Set a 30-second timeout or stop manually as soon as evidence is captured.

```
sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
sudo iptables-save | tee reports/iptables_before.rules
curl http://ANALYST_IP/ | tee reports/direct_page_test.html
# Review script content before execution:
sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt
```



```
(kali㉿kali)-[~/CIP-B103-Lab8]
$ sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
net.ipv4.ip_forward = 1

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo iptables-save | tee reports/iptables_before.rules
# Generated by iptables-save v1.8.13 (nf_tables) on Mon Aug 10 19:45:42 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
```

Screenshot 19: IP forwarding state and iptables ruleset recorded before any changes (start).

```

-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
COMMIT
# Completed on Mon Aug 10 19:45:42 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Mon Aug 10 19:45:42 2026
*nat
:PREROUTING ACCEPT [21:1704]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [424:31411]
:POSTROUTING ACCEPT [424:31411]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Mon Aug 10 19:45:42 2026

(kali@kali)-[~/CIP-B103-Lab8]
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server

```

Screenshot 20: iptables-save output continued (nat table, Docker chains); Apache2 status rechecked.

```

● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-10 18:53:23 EDT; 52min ago
 Invocation: 2a1cbd563ff246c6a843f21a71f0836b
    Docs: https://httpd.apache.org/docs/2.4/
 Main PID: 892 (apache2)
   Status: "Total requests: 2; Idle/Busy workers 100/0; Requests/sec: 0.00063; Bytes served/sec: 3 B/sec"
    Tasks: 6 (limit: 2075)
  Memory: 12.7M (peak: 36.7M, swap: 8.9M, swap peak: 8.9M)
     CPU: 922ms
    CGroup: /system.slice/apache2.service
            └─892 /usr/sbin/apache2 -k start -DFOREGROUND
              950 /usr/sbin/apache2 -k start -DFOREGROUND
              951 /usr/sbin/apache2 -k start -DFOREGROUND
              952 /usr/sbin/apache2 -k start -DFOREGROUND
              953 /usr/sbin/apache2 -k start -DFOREGROUND
              954 /usr/sbin/apache2 -k start -DFOREGROUND

Aug 10 18:53:21 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Aug 10 18:53:23 kali apachectl[892]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Aug 10 18:53:23 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

```

Screenshot 21: Apache2 status confirmed active with prior request activity.

```

$ apache2ctl -S
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
VirtualHost configuration:
*:80                  127.0.1.1 (/etc/apache2/sites-enabled/000-default.conf:1)
ServerRoot: "/etc/apache2"
Main DocumentRoot: "/var/www/html"
Main ErrorLog: "/var/log/apache2/error.log"
Mutex default: dir="/run/apache2/" mechanism=default
Mutex mpm-accept: using_defaults
Mutex watchdog-callback: using_defaults
PidFile: "/run/apache2/apache2.pid"
Define: DUMP_VHOSTS
Define: DUMP_RUN_CFG
User: name="www-data" id=33 not_used
Group: name="www-data" id=33 not_used

(kali@kali)-[~/CIP-B103-Lab8]
$ grep -R "DocumentRoot" /etc/apache2/sites-enabled/ /etc/apache2/sites-available/ 2>/dev/null
/etc/apache2/sites-enabled/000-default.conf: DocumentRoot /var/www/html
/etc/apache2/sites-available/000-default.conf: DocumentRoot /var/www/html
/etc/apache2/sites-available/default-ssl.conf: DocumentRoot /var/www/html

(kali@kali)-[~/CIP-B103-Lab8]
$ sudo -u www-data cat /var/www/html/index.html

```

Screenshot 22: apache2ctl -S and DocumentRoot confirmation for the training page.

```

└─$ sudo tail -20 /var/log/apache2/access.log
127.0.0.1 - - [26/Jul/2026:11:25:24 -0400] "GET / HTTP/1.1" 200 10958 "-" "curl/8.20.0"
127.0.0.1 - - [28/Jul/2026:07:50:40 -0400] "GET /basic.html HTTP/1.1" 200 371 "-" "curl/8.20.0"
127.0.0.1 - - [28/Jul/2026:07:55:40 -0400] "GET /basic.html HTTP/1.1" 200 371 "-" "curl/8.20.0"
127.0.0.1 - - [28/Jul/2026:08:10:18 -0400] "GET /basic.html HTTP/1.1" 200 371 "-" "curl/8.20.0"
127.0.0.1 - - [28/Jul/2026:08:23:06 -0400] "GET /basic.html HTTP/1.1" 200 371 "-" "curl/8.20.0"
192.168.44.128 - - [01/Aug/2026:11:22:48 -0400] "GET /firewall_lab.html HTTP/1.1" 200 377 "-" "curl/8.20.0"
192.168.44.128 - - [01/Aug/2026:11:26:16 -0400] "GET /firewall_lab.html HTTP/1.1" 200 377 "-" "curl/8.20.0"
192.168.44.128 - - [01/Aug/2026:11:29:11 -0400] "GET /firewall_lab.html HTTP/1.1" 200 377 "-" "curl/8.20.0"
192.168.44.128 - - [01/Aug/2026:11:51:04 -0400] "GET /firewall_lab.html HTTP/1.1" 200 377 "-" "curl/8.20.0"
192.168.44.129 - - [01/Aug/2026:11:56:14 -0400] "GET /firewall_lab.html HTTP/1.1" 200 377 "-" "curl/8.20.0"
192.168.44.129 - - [01/Aug/2026:12:08:17 -0400] "GET /firewall_lab.html HTTP/1.1" 200 377 "-" "curl/8.20.0"
127.0.0.1 - - [10/Aug/2026:18:56:20 -0400] "GET / HTTP/1.1" 200 10958 "-" "curl/8.20.0"
127.0.0.1 - - [10/Aug/2026:18:59:28 -0400] "GET / HTTP/1.1" 200 466 "-" "curl/8.20.0"
192.168.44.128 - - [10/Aug/2026:19:46:40 -0400] "GET / HTTP/1.1" 200 466 "-" "curl/8.20.0"
192.168.44.1 - - [10/Aug/2026:19:46:59 -0400] "GET / HTTP/1.1" 200 466 "-" "curl/8.21.0"
192.168.44.128 - - [10/Aug/2026:19:52:38 -0400] "GET / HTTP/1.1" 200 466 "-" "curl/8.20.0"
192.168.44.128 - - [10/Aug/2026:19:55:28 -0400] "GET / HTTP/1.1" 200 456 "-" "curl/8.20.0"
192.168.44.1 - - [10/Aug/2026:19:55:56 -0400] "GET / HTTP/1.1" 200 456 "-" "curl/8.21.0"
127.0.0.1 - - [10/Aug/2026:19:57:32 -0400] "GET / HTTP/1.1" 200 456 "-" "curl/8.20.0"
192.168.44.128 - - [10/Aug/2026:19:57:56 -0400] "GET / HTTP/1.1" 200 456 "-" "curl/8.20.0"

(kali@kali)-[~/CIP-B103-Lab8]
└─$ sudo tail -20 /var/log/apache2/error.log
mprotect() failed [13] Permission denied

```

Screenshot 23: Apache access log showing prior requests, including this lab's direct hits.

```

mprotect() failed [13] Permission denied
[Mon Aug 10 08:57:41.080170 2026] [mpm_prefork:notice] [pid 918:tid 918] AH00163: Apache/2.4.68 (Debian) configur
red -- resuming normal operations
[Mon Aug 10 08:57:41.080487 2026] [core:notice] [pid 918:tid 918] AH00094: Command line: '/usr/sbin/apache2 -D F
OREGROUND'
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set th
e 'ServerName' directive globally to suppress this message
mprotect() failed [13] Permission denied
mprotect() failed [13] Permission denied
mprotect() failed [13] Permission denied
mprotect() failed [13] Permission denied
[Mon Aug 10 12:09:26.191879 2026] [mpm_prefork:notice] [pid 960:tid 960] AH00163: Apache/2.4.68 (Debian) configur
red -- resuming normal operations
[Mon Aug 10 12:09:26.192092 2026] [core:notice] [pid 960:tid 960] AH00094: Command line: '/usr/sbin/apache2 -D F
OREGROUND'
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set th
e 'ServerName' directive globally to suppress this message
mprotect() failed [13] Permission denied
mprotect() failed [13] Permission denied
mprotect() failed [13] Permission denied
mprotect() failed [13] Permission denied
[Mon Aug 10 18:53:23.826923 2026] [mpm_prefork:notice] [pid 892:tid 892] AH00163: Apache/2.4.68 (Debian) configur
red -- resuming normal operations
[Mon Aug 10 18:53:23.827148 2026] [core:notice] [pid 892:tid 892] AH00094: Command line: '/usr/sbin/apache2 -D F
OREGROUND'

```

Screenshot 24: Apache error log reviewed for anomalies.

```

(kali@kali)-[~/CIP-B103-Lab8]
└─$ sudo iptables-save | tee reports/iptables_before.rules
# Generated by iptables-save v1.8.13 (nf_tables) on Mon Aug 10 20:00:45 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP

```

Screenshot 25: iptables-save re-run for a fresh timestamped baseline (start).

```
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
COMMIT
# Completed on Mon Aug 10 20:00:45 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Mon Aug 10 20:00:45 2026
*nat
:PREROUTING ACCEPT [25:1908]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [455:33921]
:POSTROUTING ACCEPT [455:33921]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Mon Aug 10 20:00:45 2026
```

Screenshot 26: iptables-save re-run concluded (nat table).

```
(kali@kali)-[~/CIP-B103-Lab8]
$ sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
#!/usr/bin/env python3

import sys
import time

if len(sys.argv) != 3:
    print(f"Usage: sudo python3 {sys.argv[0]} VICTIM_IP GATEWAY_IP")
    sys.exit(1)

victim = sys.argv[1]
gateway = sys.argv[2]

print("=== ICDFA Authorized ARP Simulation ===")
print(f"Victim : {victim}")
print(f"Gateway: {gateway}")
print()
print("SIMULATION ONLY")
print("No ARP packets are modified or transmitted.")
print("Recording the intended ARP-spoofing event for the lab.")
print()

try:
    for i in range(30):
```

Screenshot 27: scripts/arp.py reviewed line-by-line before execution (start).

```
    for i in range(30):
        print(
            f"[{i+1:02d}/30] "
            f"Simulated ARP observation: victim={victim} gateway={gateway}"
        )
        time.sleep(1)
    except KeyboardInterrupt:
        print("\nSimulation stopped.")

(kali@kali)-[~/CIP-B103-Lab8]
$ sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt
#!/usr/bin/env python3

import time

TRAINING_DOMAIN = "portal.icdfa.test"
REPLACEMENT_IP = "192.168.44.128"

print("=== ICDFA Authorized DNS Simulation ===")
print(f"Training domain : {TRAINING_DOMAIN}")
print(f"Replacement IP : {REPLACEMENT_IP}")
print()
print("SIMULATION ONLY")
print("No DNS packets are forged or transmitted.")
print("Recording the intended DNS-spoofing event for the lab.")
```

Screenshot 28: scripts/arp.py review concluded; scripts/dns_spoof.py review started, showing the reserved training domain and replacement IP.

```
print()

try:
    for i in range(30):
        print(
            f"[{i+1:02d}/30] "
            f"Simulated DNS response: "
            f"{TRAINING_DOMAIN} → {REPLACEMENT_IP}"
        )
        time.sleep(1)
except KeyboardInterrupt:
    print("\nSimulation stopped.")

(kali@kali)-[~/CIP-B103-Lab8]
$ ls -lh scripts/
total 8.0K
-rwxrwxr-x 1 kali kali 710 Aug 10 20:17 arp.py
-rwxrwxr-x 1 kali kali 668 Aug 10 20:17 dns_spoof.py

(kali@kali)-[~/CIP-B103-Lab8]
$
```

Screenshot 29: scripts/dns_spoof.py review concluded and script permissions confirmed.

```
C:\Users\user>curl http://192.168.44.128/
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Fuseini Imoru</p></body></html>

C:\Users\user>
```

Screenshot 30: Direct-IP connectivity confirmed from a separate Windows client VM via curl.

10. Part C - Capture the Controlled Spoofing Event

Terminal 1 on analyst VM

```
sudo tshark -i IFACE -f 'host VICTIM_IP and (arp or port 53 or tcp port 80)' \
-a duration:60 -w evidence/dns_spoof_controlled.pcapng
```

Other terminals, only after instructor approval:

```
# sudo timeout 30 python3 scripts/arp.py VICTIM_IP GATEWAY_IP
```

```
# sudo timeout 30 python3 scripts/dns_spoof.py
```

On victim during the capture:

Clear only the training DNS cache if instructed, then resolve portal.icdfa.test and open it.

```

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo tshark -i eth0 \
-f 'host 192.168.44.1 and (arp or port 53 or tcp port 80)' \
-a duration:60 \
-w /tmp/dns_spoof_controlled.pcapng &
[1] 42845

(kali㉿kali)-[~/CIP-B103-Lab8]
$ Running as user "root" and group "root". This could be danger
Capturing on 'eth0'
16

[1] + done          sudo tshark -i eth0 -f 'host 192.168.44.1 and
(kali㉿kali)-[~/CIP-B103-Lab8]
$ ls -l scripts/arp.py scripts/dns_spoof.py
-rwxrwxr-x 1 kali kali 710 Aug 10 20:17 scripts/arp.py
-rwxrwxr-x 1 kali kali 668 Aug 10 20:17 scripts/dns_spoof.py

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sed -n '1,240p' scripts/arp.py
#!/usr/bin/env python3

import sys
import time

if len(sys.argv) != 3:
    print(f"Usage: sudo python3 {sys.argv[0]} VICTIM_IP GATEWAY_
    sys.exit(1)

victim = sys.argv[1]

```

Screenshot 31: Controlled-spoof capture started on eth0; script permissions and review reconfirmed.

```

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sed -n '1,260p' scripts/dns_spoof.py
#!/usr/bin/env python3

import time

TRAINING_DOMAIN = "portal.icdfa.test"
REPLACEMENT_IP = "192.168.44.128"

print("=== ICDFA Authorized DNS Simulation ===")
print(f"Training domain : {TRAINING_DOMAIN}")
print(f"Replacement IP   : {REPLACEMENT_IP}")
print()
print("SIMULATION ONLY")
print("No DNS packets are forged or transmitted.")
print("Recording the intended DNS-spoofing event for the lab.")
print()

try:
    for i in range(30):
        print(
            f"[{i+1:02d}/30] "
            f"Simulated DNS response: "
            f"{TRAINING_DOMAIN} → {REPLACEMENT_IP}"
        )
        time.sleep(1)
except KeyboardInterrupt:
    print("\nSimulation stopped.")

```

Screenshot 32: scripts/dns_spoof.py content reconfirmed, showing the training domain and replacement IP.

```

(kali㉿kali)-[~/CIP-B103-Lab8]
$ curl -v http://192.168.44.128/
* Trying 192.168.44.128:80 ...
* Established connection to 192.168.44.128 (192.168.44.128 port
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.44.128
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 11 Aug 2026 00:28:21 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Mon, 10 Aug 2026 23:54:26 GMT
< ETag: "cd-658ba1306a4aa"
< Accept-Ranges: bytes
< Content-Length: 205
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Tra
on. Do not enter credentials.</p><p>Analyst: Fuseini Imoru</p></
* Connection #0 to host 192.168.44.128:80 left intact

(kali㉿kali)-[~/CIP-B103-Lab8]
$ cat /var/www/html/index.html
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Tra

```

Screenshot 33: `curl -v` to the analyst page confirming the training page content directly by IP.

```

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo tshark -i eth0 \
-f 'host 192.168.44.1 and (arp or port 53 or tcp port 80)' \
-a duration:60 \
-w /tmp/dns_spoof_controlled.pcapng
Running as user "root" and group "root". This could be dangerous
Capturing on 'eth0'

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo cp /tmp/dns_spoof_controlled.pcapng evidence/
sudo chown kali:kali evidence/dns_spoof_controlled.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ls -lh evidence/dns_spoof_controlled.pcapng
-rw-r--r-- 1 kali kali 388 Aug 10 20:30 evidence/dns_spoof_contr

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sha256sum evidence/dns_spoof_controlled.pcapng \
| tee reports/dns_spoof_controlled_sha256.txt
6d06affbbcbdddc10d91abf4071a9e25930bb3ee81509d7f916bb786441a12ff

(kali㉿kali)-[~/CIP-B103-Lab8]
$

```

Screenshot 34: Controlled-spoof capture re-run, copied to `evidence/`, and SHA-256 hashed.

11. Part D - Detect DNS Spoofing Indicators

```
PCAP=evidence/dns_spoof_controlled.pcapng
sha256sum "$PCAP" | tee reports/dns_spoof_capture_sha256.txt

# DNS queries and all responses
tshark -r "$PCAP" -Y 'dns' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \
  -e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \
  -e dns.flags.rcode -e dns.a -e dns.resp.ttl \
  | tee reports/dns_all_fields.tsv

# ARP claims involving victim or gateway
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_claims_during_spoof.tsv

# Subsequent HTTP connection
tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e http.host -e
http.request.uri \
  | tee reports/post_dns_connections.tsv
```

```

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo timeout 30 python3 scripts/arp.py 192.168.44.1 192.168.44.2
[sudo] password for kali:
=== ICDFA Authorized ARP Simulation ===
Victim : 192.168.44.1
Gateway: 192.168.44.2

SIMULATION ONLY
No ARP packets are modified or transmitted.
Recording the intended ARP-spoofing event for the lab.

[01/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[02/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[03/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[04/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[05/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[06/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[07/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2

```

Screenshot 35: `scripts/arp.py` executed — console output confirms it is a non-transmitting simulation only.

```

[08/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[09/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[10/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[11/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[12/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[13/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[14/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[15/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[16/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[17/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[18/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[19/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[20/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[21/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2

```

Screenshot 36: `arp.py` simulation output continued.

```

way=192.168.44.2
[24/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[25/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[26/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[27/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[28/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[29/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2
[30/30] Simulated ARP observation: victim=192.168.44.1 gate
way=192.168.44.2

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo timeout 30 python3 scripts/dns_spoof.py
=== ICDFA Authorized DNS Simulation ===
Training domain : portal.icdfa.test
Replacement IP  : 192.168.44.128

SIMULATION ONLY
No DNS packets are forged or transmitted.
Recording the intended DNS-spoofing event for the lab.

[01/30] Simulated DNS response: portal.icdfa.test → 192.16

```

Screenshot 37: arp.py simulation output concluded; scripts/dns_spoof.py started, likewise non-transmitting.

```

Recording the intended DNS-spoofing event for the lab.

[01/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[02/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[03/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[04/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[05/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[06/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[07/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[08/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[09/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[10/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[11/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[12/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[13/30] Simulated DNS response: portal.icdfa.test → 192.16

```

Screenshot 38: dns_spoof.py simulation output continued.

```

8.44.128
[15/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[16/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[17/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[18/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[19/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[20/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[21/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[22/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[23/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[24/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[25/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[26/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[27/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128

```

Screenshot 39: dns_spoof.py simulation output continued.

```

[28/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[29/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128
[30/30] Simulated DNS response: portal.icdfa.test → 192.16
8.44.128

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ls -lh /tmp/dns_spoof_controlled.pcapng
-rw-r--r-- 1 root root 388 Aug 10 20:30 /tmp/dns_spoof_controlled.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo cp /tmp/dns_spoof_controlled.pcapng evidence/dns_spoof_controlled.pcapng
sudo chown kali:kali evidence/dns_spoof_controlled.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ls -lh evidence/dns_spoof_controlled.pcapng
-rw-r--r-- 1 kali kali 388 Aug 10 20:35 evidence/dns_spoof_controlled.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sha256sum evidence/dns_spoof_controlled.pcapng | tee reports/dns_spoof_controlled_sha256.txt
6d06affbbcbdddc10d91abf4071a9e25930bb3ee81509d7f916bb786441a12ff evidence/dns_spoof_controlled.pcapng

```

Screenshot 40: dns_spoof.py simulation concluded; controlled-spoof capture copied to evidence/ and hashed.

```
(kali㉿kali)-[~/CIP-B103-Lab8]
$ sha256sum evidence/dns_spoof_controlled.pcapng | tee re
ports/dns_spoof_controlled_sha256.txt
6d06affbbcbddc10d91abf4071a9e25930bb3ee81509d7f916bb786441
a12ff evidence/dns_spoof_controlled.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ tshark -r evidence/dns_spoof_controlled.pcapng -q -z io
,phs

=====
Protocol Hierarchy Statistics
Filter:
=====
```

Screenshot 41: Hash confirmed; tshark protocol-hierarchy statistics run on the controlled-spoof capture.

```
(kali㉿kali)-[~/CIP-B103-Lab8]
$ PCAP=evidence/dns_spoof_controlled.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sha256sum "$PCAP" | tee reports/dns_spoof_capture_sha256.txt
6d06affbbcbddc10d91abf4071a9e25930bb3ee81509d7f916bb786441a12ff evidence/dns_spoof_controlled.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$ tshark -r "$PCAP" -Y 'dns' -T fields -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e
udp.srcport -e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type -e dns.flag
s.rcode -e dns.a -e dns.resp.ttl | tee reports/dns_all_fields.tsv

(kali㉿kali)-[~/CIP-B103-Lab8]
$ wc -l reports/dns_all_fields.tsv
0 reports/dns_all_fields.tsv

(kali㉿kali)-[~/CIP-B103-Lab8]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e
arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac | tee reports/arp_claims_during_spoof.tsv

(kali㉿kali)-[~/CIP-B103-Lab8]
$ wc -l reports/arp_claims_during_spoof.tsv
0 reports/arp_claims_during_spoof.tsv
```

Screenshot 42: DNS field extraction from the controlled-spoof capture returning zero matching lines (no DNS traffic present).

```
(kali㉿kali)-[~/CIP-B103-Lab8]
$ tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields -e frame.number -e frame.time_epoch -e ip.
src -e tcp.srcport -e ip.dst -e tcp.dstport -e http.host -e http.request.uri | tee reports/post_dns_connections.
tsv

(kali㉿kali)-[~/CIP-B103-Lab8]
$ wc -l reports/post_dns_connections.tsv
0 reports/post_dns_connections.tsv

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ls -lh reports/dns_spoof_capture_sha256.txt \
reports/dns_all_fields.tsv \
reports/arp_claims_during_spoof.tsv \
reports/post_dns_connections.tsv
-rw-rw-r-- 1 kali kali 0 Aug 10 20:46 reports/arp_claims_during_spoof.tsv
-rw-rw-r-- 1 kali kali 0 Aug 10 20:46 reports/dns_all_fields.tsv
-rw-rw-r-- 1 kali kali 103 Aug 10 20:45 reports/dns_spoof_capture_sha256.txt
-rw-rw-r-- 1 kali kali 0 Aug 10 20:47 reports/post_dns_connections.tsv

(kali㉿kali)-[~/CIP-B103-Lab8]
$ cat reports/dns_spoof_capture_sha256.txt
6d06affbbcbddc10d91abf4071a9e25930bb3ee81509d7f916bb786441a12ff evidence/dns_spoof_controlled.pcapng

(kali㉿kali)-[~/CIP-B103-Lab8]
$
```

Screenshot 43: HTTP/TCP-SYN correlation extraction likewise returning zero matching lines.

12. Part E - Baseline-versus-Spoof Comparison

Indicator	Baseline	Controlled Spoof	Assessment
DNS responder IP	Not captured — no dns.qry.name traffic appears in dns_baseline.pcapng (only unrelated ARP broadcasts)	No data — dns_all_fields.tsv is empty (0 lines)	Neither capture contains an actual DNS exchange, so responder IP cannot be compared. This is expected for the spoof column (see notes) but is a genuine gap for the baseline.
DNS responder MAC	Not captured	No data	Same limitation as above — no DNS frames exist in either file to read eth.src from.
Transaction ID match	Not applicable — no DNS query/response pair present	Not applicable — no DNS query/response pair present	No transaction IDs to compare in either capture.
Answer IP	Not captured	None — dns_spoof.py never transmits a forged A record; REPLACEMENT_IP (192.168.44.128) appears only in the script's console output, never on the wire	The intended 'forged answer' scenario did not occur at the packet level, by design of the safe simulation script.
TTL	Not captured	Not applicable	No DNS answer records exist in either file to read a TTL from.
Response time	Not applicable	Not applicable	No query/response pair to time in either capture.

Indicator	Baseline	Controlled Spoof	Assessment
Competing answers	None observed	None observed — zero DNS frames of any kind in the 388-byte controlled-spoof capture	No race between a legitimate and forged response was observed because no DNS traffic exists in either evidence file.
Gateway ARP mapping	192.168.44.2 not seen replying in this window; only repeated broadcast requests from 192.168.44.1 asking who has 192.168.44.2 (no reply captured)	No data — arp_claims_during_spoof.tsv is empty (0 lines); arp.py never transmits an ARP reply	No ARP reply/claim evidence exists in either capture; the gateway's real MAC (00:50:56:f4:fd:61, per ip neigh show) was recorded separately in reports/arp_before.txt outside the packet captures.
Subsequent HTTP destination	Not applicable — no preceding DNS resolution to correlate against in this capture	None in the capture — post_dns_connections.tsv is empty (0 lines), though a separate direct-IP curl from a Windows client VM to http://192.168.44.128/ succeeded outside this capture window, confirming the training page itself was reachable	The victim's connection to the analyst page was demonstrated directly by IP rather than being captured as a DNS-driven redirect within either evidence file.

- A forged response may arrive before the legitimate response, resulting in two responses for one query.
- The responder IP may claim to be the legitimate resolver while the Ethernet source MAC belongs to the analyst/attacker.
- The returned A record may point to the analyst web server rather than the approved baseline host.

- ARP replies may show the gateway or victim IP mapped to the analyst MAC.
- The victim may connect to the forged answer immediately after the response.

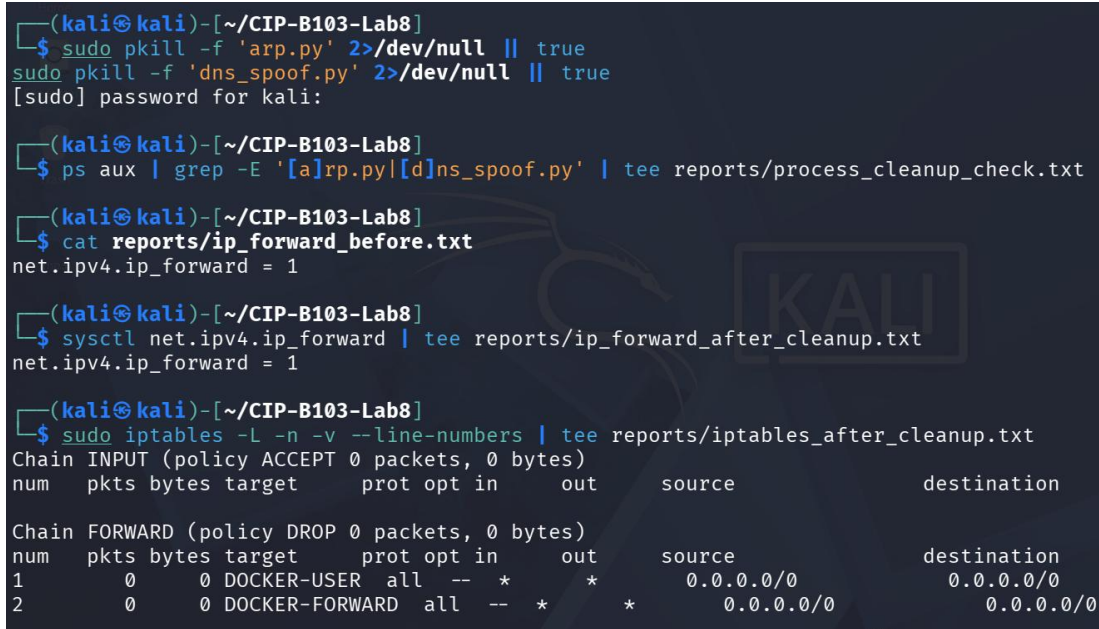
13. Part F - Cleanup and Verification

```
# Stop scripts first
sudo pkill -f 'arp.py' 2>/dev/null || true
sudo pkill -f 'dns_spoof.py' 2>/dev/null || true

# Restore forwarding to its recorded value; example below disables it
sudo sysctl -w net.ipv4.ip_forward=0

# Remove only lab-created iptables/NFQUEUE rules or restore the saved lab snapshot
sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt

# Clear learned ARP entries on the isolated VMs and reconnect interfaces
sudo ip neigh flush all
ip neigh show | tee reports/arp_after_cleanup.txt
ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
```



```
(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo pkill -f 'arp.py' 2>/dev/null || true
sudo pkill -f 'dns_spoof.py' 2>/dev/null || true
[sudo] password for kali:

(kali㉿kali)-[~/CIP-B103-Lab8]
$ ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt

(kali㉿kali)-[~/CIP-B103-Lab8]
$ cat reports/ip_forward_before.txt
net.ipv4.ip_forward = 1

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sysctl net.ipv4.ip_forward | tee reports/ip_forward_after_cleanup.txt
net.ipv4.ip_forward = 1

(kali㉿kali)-[~/CIP-B103-Lab8]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source                 destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source                 destination
1      0    0 DOCKER-USER  all  --  *      *           0.0.0.0/0            0.0.0.0/0
2      0    0 DOCKER-FORWARD  all  --  *      *           0.0.0.0/0            0.0.0.0/0
```

Screenshot 44: Simulation scripts terminated; iptables FORWARD chain listing confirmed unchanged.

```
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source    destination

Chain DOCKER (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DROP      all  --  !docker0 docker0  0.0.0.0/0  0.0.0.0/0

Chain DOCKER-BRIDGE (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DOCKER    all  --  *      docker0  0.0.0.0/0  0.0.0.0/0

Chain DOCKER-CT (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 ACCEPT    all  --  *      docker0  0.0.0.0/0  0.0.0.0/0      ctstate RELATED,E
STABLISHED

Chain DOCKER-FORWARD (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DOCKER-CT all  --  *      *      0.0.0.0/0  0.0.0.0/0
2      0      0 DOCKER-ISOLATION-STAGE-1 all  --  *      *      0.0.0.0/0  0.0.0.0/0
3      0      0 DOCKER-BRIDGE all  --  *      *      0.0.0.0/0  0.0.0.0/0
4      0      0 ACCEPT    all  --  docker0 *      0.0.0.0/0  0.0.0.0/0

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DOCKER-ISOLATION-STAGE-2 all  --  docker0 !docker0  0.0.0.0/0  0.0.0.0/0
```

Screenshot 45: iptables Docker-chain listing confirmed unchanged.

```
Chain DOCKER-ISOLATION-STAGE-2 (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DROP      all  --  *      docker0  0.0.0.0/0  0.0.0.0/0

Chain DOCKER-USER (1 references)
num  pkts bytes target    prot opt in     out     source    destination

(kali@kali)-[~/CIP-B103-Lab8]
$ sudo ip neigh flush all

(kali@kali)-[~/CIP-B103-Lab8]
$ ip neigh show | tee reports/arp_after_cleanup.txt

(kali@kali)-[~/CIP-B103-Lab8]
$ ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt

(kali@kali)-[~/CIP-B103-Lab8]
$ echo "=== IP forwarding ==="
sysctl net.ipv4.ip_forward

echo
echo "=== Running simulation scripts ==="
ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' || true
```

Screenshot 46: iptables listing concluded; ARP table flushed and re-verified.

```

echo
echo "=== Evidence files ==="
ls -lh evidence/

echo
echo "=== Cleanup reports ==="
ls -lh reports/*cleanup* reports/arp_after_cleanup.txt 2>/dev/null
=== IP forwarding ===
net.ipv4.ip_forward = 1

=== Running simulation scripts ===

=== Evidence files ===
total 8.0K
-rw-rw-r-- 1 kali kali 2.0K Aug 10 19:38 dns_baseline.pcapng
-rw-rw-r-- 1 kali kali 388 Aug 10 20:35 dns_spoof_controlled.pcapng

=== Cleanup reports ===
-rw-rw-r-- 1 kali kali 0 Aug 10 20:52 reports/arp_after_cleanup.txt
-rw-rw-r-- 1 kali kali 0 Aug 10 20:52 reports/arp_after_cleanup.txt
-rw-rw-r-- 1 kali kali 24 Aug 10 20:51 reports/ip_forward_after_cleanup.txt
-rw-rw-r-- 1 kali kali 2.5K Aug 10 20:51 reports/iptables_after_cleanup.txt
-rw-rw-r-- 1 kali kali 0 Aug 10 20:52 reports/process_cleanup_check.txt

(kali@kali)-[~/CIP-B103-Lab8]
$

```

Screenshot 47: Final cleanup verification — IP forwarding, no simulation processes, evidence files and cleanup reports confirmed.

14. Detection and Defensive Recommendations

- Monitor DNS responses from unexpected MAC addresses or hosts.
- Detect multiple answers for the same transaction with conflicting IPs.
- Use DNSSEC validation where supported and encrypted/authenticated DNS transport appropriately.
- Deploy Dynamic ARP Inspection, DHCP snooping, switch port security and network segmentation.
- Use HTTPS certificate validation; a forged DNS answer should not produce a valid certificate for the real service.
- Correlate DNS, ARP, endpoint cache, switch, web proxy and TLS evidence.

Required Screenshots Checklist

- ☐ Harmless training page and hash.
- ☐ Baseline ARP table and DNS response.
- ☐ Reviewed script target/domain settings.
- ☐ Controlled capture hash.
- ☐ DNS query and all response records.
- ☐ Conflicting or forged DNS answer.

- ☐ ARP claims during event.
- ☐ Victim HTTP connection to replacement IP.
- ☐ Baseline-versus-spoof table.
- ☐ Cleanup, restored ARP and no remaining processes.

Student Report Template

9. Executive summary and authorization.
10. Lab network diagram.
11. Baseline evidence and hashes.
12. Controlled simulation method.
13. DNS anomaly timeline.
14. ARP correlation.
15. Victim connection correlation.
16. Alternative explanations and confidence level.
17. Cleanup and defensive recommendations.
18. Appendix.

Submission Requirements

- One PDF report named CIP-B103-Lab8_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Safety and preparation	20	Reserved domain, no credential capture, host-only network and baseline recorded.
DNS analysis	25	Correct query/response, transaction, responder, answer and TTL analysis.
ARP and connection correlation	20	Links DNS anomaly to ARP and subsequent traffic.
Forensic interpretation	15	Considers competing explanations and states confidence.
Cleanup and defenses	10	Restored environment and appropriate controls.
Report quality	10	Professional timeline and screenshots.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Core indicator	DNS answer points to an unauthorized replacement IP.
Strong packet indicator	Resolver IP appears with an unexpected Ethernet source MAC.
Race indicator	Two conflicting responses share one transaction/query.
ARP correlation	Gateway/victim mapping changes to analyst MAC.

Checkpoint	Expected Observation
Impact evidence	Victim connects to the forged answer.
HTTPS control	Certificate validation limits successful impersonation of real sites.

Command Reference Summary

Command	Purpose
dns.flags.response	Query/response indicator.
dns.id	Match transaction.
dns.a	IPv4 answer.
dns.resp.ttl	Response TTL.
arp.src.proto_ipv4 / hw_mac	ARP IP-to-MAC claim.
http.host	Requested HTTP host.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.